

Пермский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

Факультет социально-экономических и компьютерных наук

Берсенёв Илья Иванович
**Разработка системы для быстрого декодирования видео в
формате AV1**

Выпускная квалификационная работа
бакалавра по направлению 09.03.04 разработка информационных систем
для бизнеса

Руководитель
Старший преподаватель, научный
руководитель кафедры информационных
технологий в бизнесе, академический
руководитель образовательной
программы разработка информационных
систем для бизнеса

_____Ланин Вячевлав Владимирович

Пермь 2025

АННОТАЦИЯ

Берсенёв Илья Иванович, Разработка системы для быстрого декодирования видео в формате AV1 : исследовательская работа бакалавра 09.03.04 программная инженерия / Берсенёв Илья Иванович. – Пермь: НИУ ВШЭ – Пермь, 2025. "страниц" с.

Работа посвящена разработке информационной системы для ускорения декодирования видеофайлов закодированных в формате AV1 и исследованию способов оптимизации.

Во «Введении» обосновывается актуальность данной работы, ставятся цель проекта и задачи.

В первой главе приведены результаты анализа предметной области, в частности обзору стандарта AV1 Bitstream & Decoding Process Specification v1.0.0-errata и его реализации в существующем видеодекодере libaom. Глава заканчивается анализом требований к данной системе.

Во второй главе представлены результаты проектирования данной системы. Выполнено проектирование системы в общем и отдельных её компонентов в частности. Задачи проектирования решены на основе объектно-ориентированного подхода, при построении моделей использован язык UML.

Третья глава описывает процесс реализации системы и оптимизации видеодекодера libaom с указанием результатов оптимизации.

В «Заключении» описаны сделанные выводы и результаты работы.

Работа содержит ??? страниц основного текста, ?? таблиц, ?? рисунков. Библиографический список включает ?? наименований. Работа включает ?? приложений.

(части которые будут позже изменены или удалены выделены в работе оранжевым цветом)

ОГЛАВЛЕНИЕ

гlossарий	4
Введение	5
1. АНАЛИЗ СПЕЦИФИКАЦИИ AV1 и ТЕХНИЧЕСКИХ СРЕДСТВ ПОВЫШЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ	7
1.1. Обоснование актуальности и вывод первичных требований	7
1.2. Анализ существующих решений	7
1.3. Выделение функциональных требований	8
1.4. Определение вариантов использования системы	8
1.5. Выделение нефункциональных требований	8
1.6. Анализ стандарта AV1-v1.0.0errata	9
1.6.1. Сравнение с стандартом ITU-T H.264	9
1.6.2. Intra декодирование	11
1.6.3. Inter декодирование	13
1.6.4. Результаты анализа стандарта AV1	14
1.7. Анализ методов оптимизации кода	14
1.7.1. Правила бентли	14
1.7.2. Оптимизация CPU-кеша	15
1.8. Анализ технологического стека	16
1.9. Анализ существующей инфраструктуры	17
1.10. Результаты анализа объекта автоматизации	18
2. Проектирование	19
2.1. Проектирование наивной реализации системы	19
2.1.1. Проектирование базы данных	20
2.2. Проектирование оптимизированной реализации системы	21
2.2.1. Проектирование реляционной базы данных	24
2.2.2. Проектирование бэкенда	25
2.2.3. Проектирование фронтенда	28
3. реализация	34
1.1. Оптимизации Libaom	34
3.1.1. Оптимизация 1	34
3.1.2. Оптимизация 2	34
3.1.3. Оптимизация 20	34
3.2. Реализация backend	34
3.3. Реализация frontend	34
3.4. Результаты оптимизации	34
Заключение	35
Библиографический список	36

ГЛОССАРИЙ

Motion Vectors (MV) – метод сжатия видео путем извлечения из видео векторов движения.

Constrained directional enhanced filter (CDEF) – метод постобработки видео для устранения артефактов.

Deblocking filter (DF) – метод постобработки видео для уменьшения блочности.

Оверхед – накладные расходы какой-либо технологии.

Видеокодек (кодек) – стандартизированный метод кодирования и реализованный видео.

Видеодекодер (декодер) – стандартизированный метод декодирования видео, реализованный в виде программного обеспечения.

TAPe – запатентованная система компании «ООО КОМЭКСП», предназначенная для индексации видео.

AOMedia Video 1 (AV1) – открытый стандарт видеокодека, разработанный консорциумом AOMedia Alliance for Open Media.

Errata (AV1-v1.0.0errata) – название стандарта, определяющего поведение видеокодека AV1.

Frame – кадр в декодируемом видео, представляющий из себя последовательность пикселей. Не обязательно присутствует в конечном видео после декодирования.

Макроблок – сущность, объединяющая квадратную область из нескольких пикселей. Изначально термин из стандарта ITU-T H.264, но также будет использоваться для обозначения суперблоков из стандарта Errata.

Slice – сущность, объединяющая макроблоки.

ВВЕДЕНИЕ

Индустрия кодирования видео традиционно является одной из самых сложных среди других и демонстрирует впечатляющие результаты. К примеру, видеокодек H.264, чья первая версия стандарта была выпущена в 2003 году, демонстрирует снижение веса видео в 50-500 раз в сравнении с последовательно закодированными в png кадрами.¹

Чтобы доказать данное утверждение проведем небольшой мысленный эксперимент: пусть у нас есть пятиминутное трехцветное видео с разрешением 1920*1080 и частотой кадров 30 в секунду. Тогда его размер будет равен $1920*1080*(3*1)*30*(5*60)=55\,987\,200\,000$ байт, что приблизительно равно 52 гигабайтам [9]. Очевидно, что любой видеодекодер сжимает видео лучше¹.

Такая внушительная эффективность достигается благодаря тщательно продуманным стандартам с учетом архитектуры современных процессоров, математическим оптимизациям в области линейной алгебры, технологии кодирования motion vectors и способам предсказаний следующих кадров, и с каждым новым стандартом кодера эффективность лишь повышается.

Однако у такой высокой эффективности сжатия существует и обратная сторона, которая заключается в том, что с повышением уровня сжатия также повышается и необходимая процессорная нагрузка для декодирования файла.

Данная выпускная квалификационная работа решает реальную бизнес-задачу для компании ООО «КОМЭКСП» (<https://comexp.net/>). Основной продукт данной компании – запатентованный индексатор видео TAPe, который позволяет осуществлять поиск видео по видео. Проблема заключается в том, что TAPe работает сильно быстрее любого видеодекодера. Так, например, декодирование двухчасового видеофильма «Форсаж 8» занимает три минуты, а его индексирование происходит за десять секунд, в результате чего индексация видео занимает лишь пять процентов от общего времени работы системы.

Ввиду вышеперечисленных обстоятельств было принято решение разработать систему чтобы максимально уменьшить простой индексатора относительно AV1 видео.

Данная работа будет выполняться в следующих условиях: процессор Intel e5-2697 v2, SSD, однопоточный режим, выходное разрешение 64px*64px пропорционально входному. Данный выбор обусловлен тем, что однопоточные оптимизации легче переносить на другие среды выполнения, они являются легковесными и могут запускаться на клиенте, а также потребляют меньше процессорных ресурсов чем многопоточная среда. CUDA использована не будет так как её реализация будет выполнена в будущем и будет основываться на оптимизациях в текущей работе. Сверхнизкое выходное разрешение обусловлено особенностями работы TAPe, о чем будет более подробно упомянуто в первой главе.

Объект исследования – процесс декодирования видео в формате AV1.

¹ Стоит, однако упомянуть формат GIF, который хранит кадры именно как последовательность изображений, из-за чего неэффективен по сравнению с любым видеокодеком.

Предмет исследования – алгоритмы и программные методы повышения производительности декодирования видео в формате AV1.

Цель исследования – максимально сократить процесс «загрузка-декодирование» для видео в формате AV1.

Для достижения указанной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и базируясь на основе методов оптимизации ПО разработать требования к системе декодирования видео в формате AV1.
2. Провести проектирование архитектуры системы согласно требованиям.
3. Провести реализацию системы.
4. Провести развертывание системы.

Практическая значимость данной работы заключается в сокращении времени декодирования видео, что уменьшит время простоя индексатора TAPe, увеличит быстродействие системы и удешевит масштабирование продуктов, основанных на TAPe.

1. АНАЛИЗ СПЕЦИФИКАЦИИ AV1 И ТЕХНИЧЕСКИХ СРЕДСТВ ПОВЫШЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ

В ходе анализа должны быть решены следующие задачи:

1. Выведение функциональных и нефункциональных требований к системе
2. Поиск потенциальных узких мест в стандарте Errata
3. Анализ методов оптимизации кода с целью их дальнейшего использования
4. Выбор оптимального технического стека для реализации системы

1.1. Обоснование актуальности и вывод первичных требований

Компания «ООО КОМЭКСП», ради которой выполняется работа, оказывает услуги по поиску видео по видео с помощью технологии индексации TAPe. TAPe принимает на вход кадры видео, а на выходе выдает «индекс» видео, представляющий из себя сжатый набор черт по которым видео можно отличать друг от друга и присваивать им индекс «похожести», именуемый T-индексом.

Сам процесс индексации происходит относительно быстро и ограничен по большей части скоростью чтения данных, однако декодирование видео с целью получения кадров из потока байтов происходит сильно медленнее. Например, индексация фильма «Форсаж 8» весом в 3 гигабайта в разрешении 1920*1080 занимает 10 секунд, а его декодирование стандартным видеodecoderом libaom – 247 секунд.

Из вышесказанного следует, что имеет смысл оптимизировать именно процесс передачи видео на сервер и его последующее декодирование, так как именно он является «бутылочным горлышком» для всего процесса индексации.

Таким образом, к системе можно предъявить два важных первичных требования: возможность параллельно обрабатывать множество запросов, а также максимально сократить время, которое необходимо для того чтобы передать видео на сервер и декодировать его.

Система будет использовать видеodecoder Libaom для декодирования AV1 видео и он должен быть собран под архитектуру generic. Это значит, что в нем не будут использоваться процессор-специфичные оптимизации вроде SIMD, AVX, SSE2 для облегчения дальнейшего портирования. Также не будет использоваться Nvidia CUDA ввиду того что в данной работе первостепенной целью является алгоритмическая оптимизация, а не «железная».

При этом неважно качество работы системы на видео высокого разрешения, TAPe одинаково хорошо работает для видео 128*128 и для видео 1920*1920, так что оптимизации будут проводиться с потерей качества.

1.2. Анализ существующих решений

Система имеет в требованиях возможность полного декодирования видео в формате AV1 как можно быстрее, притом необязательно чтобы видео в высоком разрешении имело высокое выходное качество. Из этих двух требований следует что не может быть аналогов данной системы (по крайней мере, в открытом доступе) ввиду того, что пользователям в абсолютном большинстве случаев не нужно

декодировать всё видео сразу так как большинство видеodeкодеров поддерживают потоковое и покадровое декодирование. Ситуация осложняется тем, что к каждому из видеodeкодеров необходимо подбирать свой метод оптимизации, что достаточно трудоемко.

Можно возразить, что на видеохостингах вроде youtube пользователи часто выбирают видео низкого разрешения чтобы сэкономить мобильный трафик, однако это означает что на их устройства просто приходит изначально закодированное в низком разрешении видео, а данная система декодирует изначально видео в высоком качестве в низкое чтобы увеличить скорость.

1.3. Выделение функциональных требований

Для системы быстрого декодирования видео в формате AV1 были сформулированы функциональные требования:

- Система должна декодировать видео в формате AV1
- Система должна иметь возможность параллельно обрабатывать несколько запросов
- Система должна иметь возможность загрузки видео пользователем
- Система должна иметь возможность авторизации пользователем

1.4. Определение вариантов использования системы

Система будет иметь единственное предназначение, возможность провести видео через процесс «загрузка-декодирование» максимально быстро. Для этого процесса также необходима авторизация для идентификации пользователя. В приложении А представлена диаграмма последовательностей для этого варианта использования системы. В приложении Б представлена диаграмма прецедентов использования системы.

Имеет необходимость пояснить термин «obu» в данной диаграмме: согласно AV1-v1.0.0errata obu расшифровывается как open bitstream unit и является неким блоком, из которых состоит видео. Тут, однако, имеется файл с расширением .obu, коим является видеодорожка, отделенная от аудио. Это одна из оптимизаций, которая будет объяснена и применена в главах 2 и 3.

1.5. Выделение нефункциональных требований

Для системы быстрого декодирования видео в формате AV1 были сформулированы нефункциональные требования:

- Нагрузка при обработке нескольких запросов должна масштабироваться чтобы снизить задержки
- Система должна быть максимально оптимизированной
- Декодер AV1 должен быть собран под архитектуру generic и не использовать Nvidia CUDA

1.6. Анализ стандарта AV1-v1.0.0errata

В данной главе будет описан стандарт Errata с целью выявления потенциальных узких мест для оптимизации. Будет использовано сравнение с стандартом ITU-U H.264 для более наглядного объяснения, так как методы в последнем во многом служат опорой для Errata, однако более просты в понимании.

1.6.1. Сравнение с стандартом ITU-T H.264

Несмотря на то, что один из старейших стандартов кодека для H.264, появившийся в 2003 году, вышел раньше первого стандарта для AV1 на 15 лет (первая версия стандарта AV1 была опубликована в 2018 году), эти стандарты имеют много схожих черт.

На рисунке 1 представлена базовая высокоуровневая схема декодирования по стандарту H.264. На нем можно увидеть, что декодирование может быть циклическим ввиду того что некоторые кадры используют ранее декодированные кадры при реконструкции, а также что данные проходят через несколько развилок – способов предсказания [10].

Способы предсказания делятся на две важные категории: Intra предсказание и Inter предсказание. Первый метод – внутрикадровое предсказание, второе – межкадровое.

Также как в ITU-T H.264, в AV1-v1.0.0errata сохранилось intra предсказание, но стало более комплексным. В H.264 направление intra предсказания ограничено девятью углами от 216 градусов до 18 (см. рис.2, рис.3) и действует в рамках макроблоков (см. главу 1.2.2), в AV1 базовые концепты Intra предсказания сохраняются но действуют рекурсивно в рамках объединений макроблоков размерности 8×8 , которые разбиваются внутри себя на блоки размерности 4×2 . Эта оптимизация улучшает качество изображения и обеспечивает лучшую многопоточную производительность ввиду независимости 8×8 блоков но замедляет скорость декодирования.

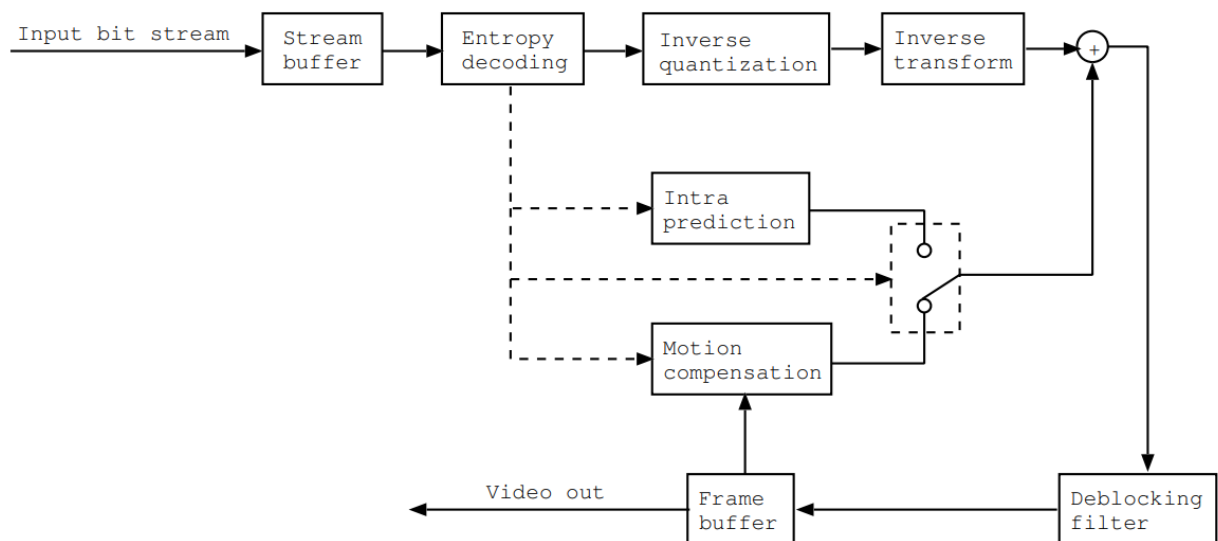


Рисунок 1 – последовательность декодирования

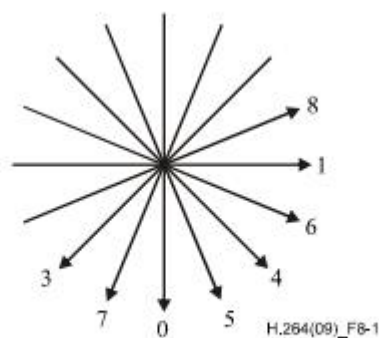


Figure 8-1 – Intra_4x4 prediction mode directions (informative)

Рисунок 2 – intra углы [2]

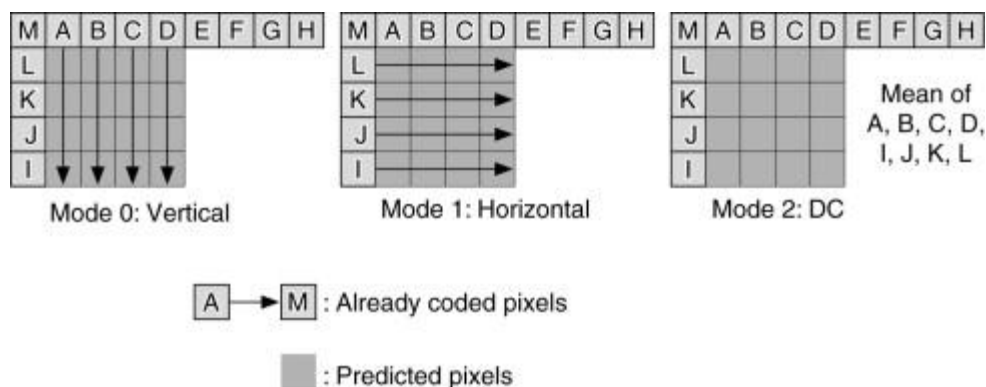


Рисунок 3 – intra предсказание [2]

Касательно Inter предсказания – в H.264 Этот способ работает так: существуют фреймы, являющиеся по сути кадрами видео. Фреймы делятся на несколько категорий, наиболее часто встречающимися являются I, P, B фреймы. I-фреймы независимо закодированы только intra методом, P-фреймы закодированы как разность motion-векторов (см. главу 1.2.3) с предыдущим кадром, B-фреймы закодированы как разность motion-векторов с предыдущим и следующим кадром. AV1 не отделяет I-фреймы от B,P-фреймов, кодируя отдельный фрейм обоими способами, использует сильно более сложные методы предсказания и имеет новый вид фреймов – golden frame. Этот фрейм содержит в себе изображение высокого качества и часто используется для реконструкции. Также в AV1 вводится понятие суперфреймов, содержащих в себе несколько других фреймов, а один кадр может ссылаться не на 1-2 фрейма как в H.264, а иметь древовидную и комплексную структуру ссылок до 8 кадров, где каждому кадру соответствует временной слой (см. рис. 4).

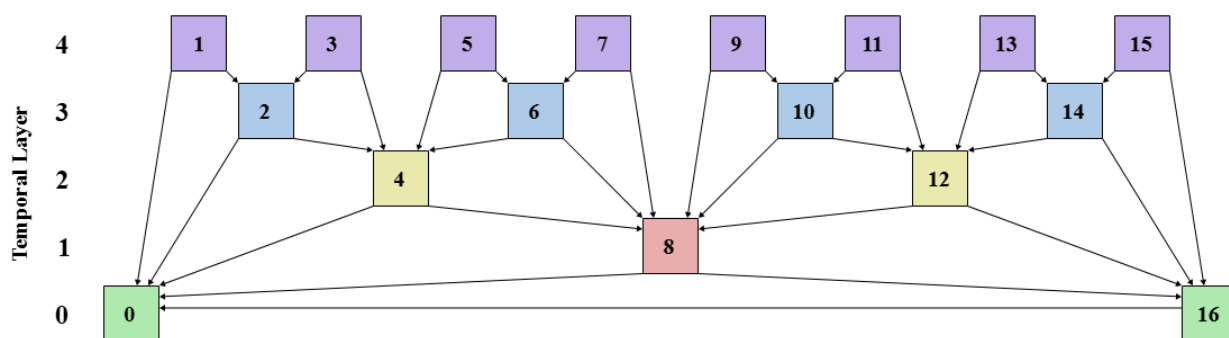


Рисунок 4 – мультиреференсные кадры в intra-предсказании [11]

1.6.2. Intra декодирование

Как уже было сказано, предсказания в кодеках делятся на 2 вида. В этой главе будут рассмотрены методы внутрикадрового предсказания.

Макроблоки

В стандарте ITU-T H.264 макроблоками называются массивы luma пикселей 16*16 и соответствующие им массивы Cb, Cr пикселей. Luma в данном случае означает яркость, Cb и Cr – цветовые компоненты синего и красного оттенков в формате YCbCr (Chroma-Luma). Макроблок – объект, которым оперирует inter или intra предсказание, а также множественные методы реконструирования и энтропийного декодирования.

В стандарте AV1-v1.0.0errata макроблоки заменяются суперблоками, однако общие принципы остаются теми же за исключением того что размерность суперблоков – 64*64.

Интерполяция

Интерполяция по своей сути в AV1-v1.0.0errata похожа на ту, что была в ITU-T H.264.

При расчете смещения пикселей с вычислением motion vectors может оказаться так, что смещение будет не на натуральное число пикселей, а на вещественное, и в этом случае результирующий пиксель рассчитывается как взвешенная сумма окружающих его пикселей. На рис. 5 представлен фрагмент из стандарта ITU-T H.264, поясняющий как именно вычислять интерполированный пиксель. Так, например, для пикселя в позиции b результирующее значение будет вычислено как $(E - 5 * F + 20 * G + 20 * H - 5 * I + J) / 32$, а результирующее значение h – $(A - 5 * C + 20 * G + 20 * M - 5 * R + T) / 32$.

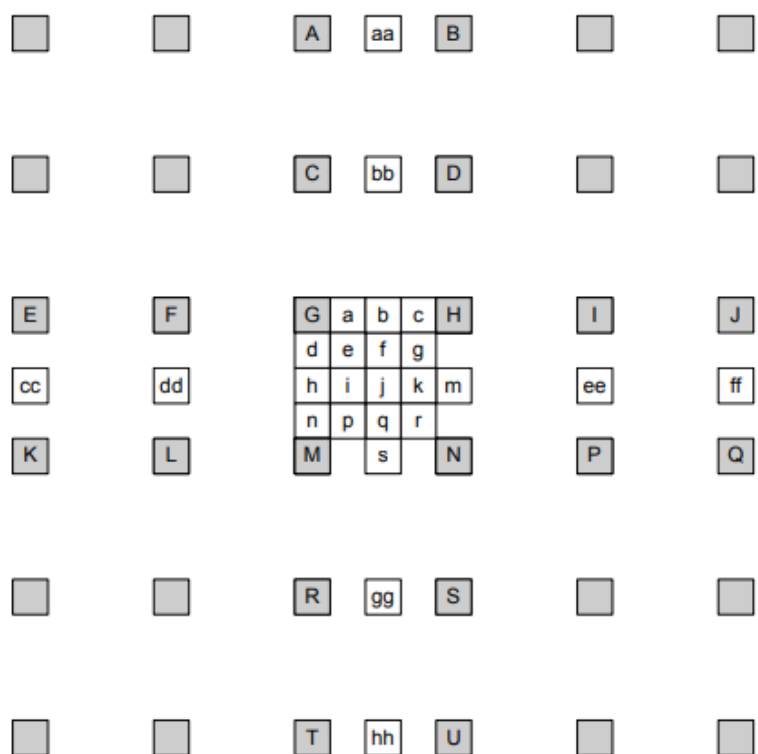


Figure 8-4 – Integer samples (shaded blocks with upper-case letters) and fractional sample positions (un-shaded blocks with lower-case letters) for quarter sample luma interpolation

Рисунок 5 – пояснение к методу вычисления интерполяции [2]

Нетрудно догадаться, что это крайне тяжелое и массовое вычисление, которое даже при максимальных оптимизациях с сохранением логики будет потреблять огромное количество процессорных тактов ввиду того, что даже для вычисления интерполяции всего лишь в одной плоскости понадобится минимум **71 такт** на x86 процессоре (не считая simd оптимизаций). Кроме 6-tap фильтра, описанного выше, существует обычная билинейная интерполяция и выбор первого или второго зависит от закодированной точности пикселя. **Вставить схему из mdl языка которую делал на морисе позже.**

В стандарте AV1 интерполяция реализована похоже, однако коэффициенты (-1, 5, 10) динамические и закодированы в видео, притом всего вариантов коэффициентов 96 – по 16 на каждый метод интерполяции. В их числе обычная интерполяция, резкая, плавная и пр. Также в AV1 выше точность интерполяции и помимо 6-tap и билинейной интерполяции существуют 4-tap и 8-tap фильтры. Данная реализация на порядок «дороже» реализации H.264.

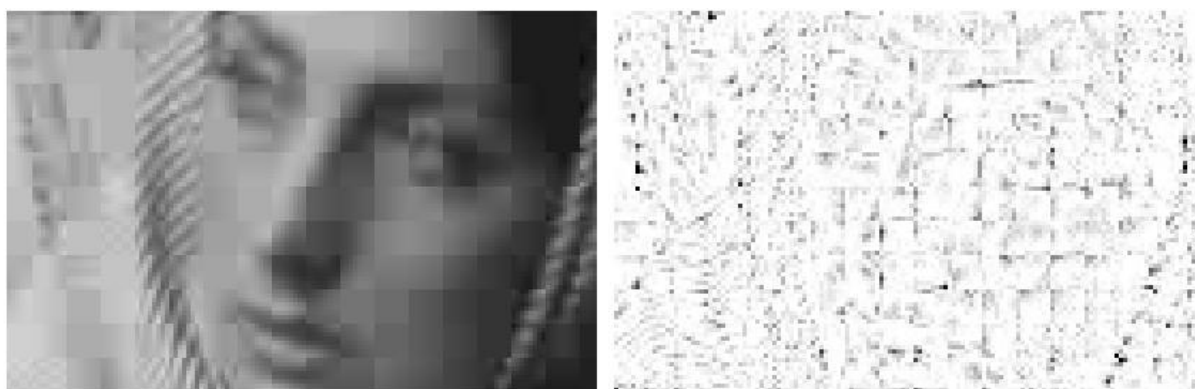
Энтропийное кодирование

В декодерах-наследниках MPEG (к коим относятся рассматриваемые AV1 и H.264) также присутствуют методы сжатия путем энтропийного кодирования вроде DCT, ADST, FLIPADST, IDTX. Не вдаваясь в детали, это методы кодирования, при котором видеodeкодер берет из потока байтов 2 коэффициента которые являются натуральными числами от 0 до 7, а затем переводит их в частоты через функцию косинуса (для discrete cosine transform). Так, например, из коэффициентов (0,0) мы

восстановим однородно-серую текстуру, из (1,0) – горизонтальный градиент, из (7,7) – мелкую сетку. Это позволяет сильно сэкономить занимаемое место и время декодирования, так как методы декодирования сильно оптимизированы даже в виде нативного кода на си.

CDEF

В AV1 существует такая технология как constrained directional enhancement filter (CDEF). CDEF представляет из себя фильтр пост-обработки, который наряду с deblocking filter (DF) улучшает качество изображения. Выше мы рассмотрели методы энтропийного кодирования, однако у них есть неприятное побочное действие: накопление ошибок на краях блоков. Восстановленная таким образом картинка похожа на мозаику из квадратов, как на рис. 6.



Closeup of reconstructed image

Normalized error distribution within each block

Рисунок 6 – ошибки инверсной дискретной реконструкции

Для решения этой проблемы существуют 3 фильтра, применяемых в каждом кадре по порядку: DF, CDEF, loop filter. Конкретно CDEF отвечает за то, чтобы убрать шум рядом с границами. Для этого вычисляется, насколько текущий блок пикселей попадает в каждый из 8 фильтров направлений (45° , 22.5° , 0° , -22.5° , -45° , -67.5° , -90° , -112.5°) и дальше по границам идет фильтр, похожий на интерполяционный.

Deblocking filter

Наряду с CDEF, DF устраняет блочность изображения, также за счет сложной интерполяции. Пока что запомним что DF и CDEF – дорогие операции для улучшения качества изображения в высоком разрешении.

1.6.3. Inter декодирование

Motion Vectors

Motion vectors (MV) – один из важнейших способов сжатия в видеодекодерах. В видео почти всегда присутствует движение, и ввиду этого вместо того чтобы кодировать каждый кадр как изображение, мы можем закодировать движение пикселей или областей пикселей. К примеру, у нас есть видео в котором мяч летит в ворота, а камера неподвижна. Вместо того, чтобы отдельно кодировать каждый кадр мы кодируем начальный кадр, а дальше – только дельты пикселей мяча для каждого кадра. Опорные кадры (также называющиеся reference frames или IDR frames) –

кадры, которые невыгодно кодировать с помощью motion vectors, например смена сцены или положения камеры.

Frames

О фреймах было рассказано в главе 1.2.1, можно лишь добавить что в AV1 существуют hidden frames – опорные фреймы, которые не добавляются в итоговый результат.

1.6.4. Результаты анализа стандарта AV1

После проведенного анализа мы можем выделить следующие средства, на которые стоит обратить внимание при оптимизации:

- CDEF
- DF
- Интерполяция
- Энтропийное кодирование

Данные операции занимают много ресурсов и должны быть оптимизированы в первую очередь. В перспективе это даст хорошее увеличение производительности.

1.7. Анализ методов оптимизации кода

В данной главе будет рассказано об основных методах оптимизации с опорой на «правила оптимизации», предложенные профессором Массачусетского Технологического Университета Джоном Луисом Бентли. В главе «Реализация» мы постараемся применить их чтобы сократить время декодирования видео. **Скорее всего бентли их в 2012 придумал но я не уверен.**

1.7.1. Правила бентли

Правила Бентли представляют из себя подобие чек-листа из 20 пунктов, разделённых на 4 раздела: структуры данных, логика, циклы и функции. Некоторые пункты потеряли актуальность в связи с тем что GNU GCC компилятор применяет их при включенных флагах -O2, -O3, а некоторые неприменимы к данной задаче. Рассмотрим оставшиеся.

Инициализация во время компиляции

Данный метод представляет из себя практику инициализации значений во время компиляции. К примеру, у нас есть функция, вычисляющая синус, умноженный на косинус, а точность выходного значения не сильно важна. В этом случае мы можем инициализировать массив вида `uint32_t cossin[10]={0.0, 0.476, 0.294, -0.294, -0.476, -0.0, 0.476, 0.294, -0.294, -0.476}`, затем вычислять приблизительное значение как `result = cossin[i]`, где $i - (\pi * 2 / 10) * (\text{требуемый угол})$. Вычисляемые значения будут с погрешностью входного значения $< \pi * 1/5$.

Плотность

Как будет рассмотрено в главе 1.3.2, крайне важно оптимизировать код для процессорного кеша, ввиду этого необходимо соблюдать плотность данных. К примеру, представим что в примере выше мы инициализировали наш массив с точностью входного значения менее $\pi * 1/500$, тогда в массиве у нас будет 1000 значений. Если код будет запрашивать элементы в соответствии с нормальным

распределением и если учесть что линия процессорного кеша вмещает 8 его элементов, то процент промахов может достигнуть 87.5%. Если же элементов 10, то при прочих равных процент промахов не будет выше 20%.

Объединение тестов

Общеизвестно что процессор имеет механизм branching. Это значит, что процессор может предсказать результат логического ветвления в коде и в соответствии с ним «положить» в конвейер инструкций соответствующий участок кода. Чем меньше процент верно угаданных развилок, тем ниже производительность, и соответственно выгоднее всего сократить количество возможных развилок объединив несколько условий в одно.

Упорядочивание проверок

В языках программирования логические операторы, как правило, «ленивые», следовательно необходимо упорядочивать проверки в условиях так, чтобы конечный результат был известен как можно раньше.

Бит хаки

Данный метод не входит в перечень оптимизаций Бентли, однако его можно отнести к подвиду объединения тестов. Суть бит хаков в том, чтобы заменить обычные логические вычисления быстрыми битовыми вычислениями.

Например, условие `if (x<4096 && x>=256)` можно заменить условием `if((x|~0xFF)&&!(x&~0xFFF))`.

Несмотря на миф о том, что современные процессоры автоматически преобразуют логические инструкции в наиболее эффективные бит хаки, это не так и преобразование условий в бит хаки эффективно. **Это будет позже доказано экспериментально.**

1.7.2. Оптимизация CPU-кеша

CPU-кеш – один из главных методов оптимизации кода. Его правильное использование может уменьшить скорость выполнения кода на величину до 99% в синтетических задачах. На рисунке 7 мы можем видеть относительную скорость доступа к разным типам памяти, и мы можем увидеть, что доступ к ячейке оперативной памяти DDR4 в 120 раз более долгий чем доступ к ячейке L1 процессорного кеша. Следовательно, максимальная выгода в синтетической задаче от перемещения данных из dram в L1 кеш более 99%.

Разумеется, это верно только для нескольких килобайт данных (48кб для intel i5-12400f), так что необходимо особо тщательно подходить к порядку запроса данных и количеству запрошенных данных.

Table 2.2 Example Time Scale of System Latencies

Event	Latency	Scaled
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access (DRAM, from CPU)	120 ns	6 min
Solid-state disk I/O (flash memory)	50–150 μ s	2–6 days
Rotational disk I/O	1–10 ms	1–12 months

*Рисунок 7 – memory latency table***Offset**

Тут про то как вычисляется адрес ячейки в кеше. Надеюсь не устаревшая информация

Cache-sparsity

Про кеш локальность с примерами.

Кеш-промахи

Про кеш-промахи соответственно. Их 4 штуки: по емкости, холодный, конфликтный, совместный (MESI).

1.8. Анализ технологического стека

Для того, чтобы построить быструю и правильно работающую систему, которую легко поддерживать необходимо выбрать подходящие технологии. В данной главе будут перечислены основные технологии необходимые для реализации проекта.

1.8.1. Веб-сервер

В качестве веб-сервера был выбран фреймворк **FastApi**. Несмотря на то, что он написан на Python и в связи с этим имеет проблемы с Requests Per Second, он является одним из самых быстрых веб-фреймворков для Python [15] ввиду минимального оверхеда, в отличие например от Django с большим количеством слоев абстракций и синхронной блокирующей архитектурой.

1.8.2. Обработка сетевых пакетов

Для обработки сетевых пакетов видео будет использоваться модуль **DPDK**. Он позволяет обрабатывать пакеты напрямую в user-space, минуя уровень ядра ОС linux.

(не уверен оставлю ли в финале, звучит достаточно сложно. Реализую если останется время)

1.8.3. Оркестрация

Учитывая требование о масштабируемости системы, она будет расположена на нескольких серверах для того чтобы обеспечить линейную загрузенность пропорционально запросам. В связи с этим необходимо оркестрировать серверы, и для этого лучше всего подходит **Kubernetes** ввиду его популярности и хорошей документации.

1.8.4. Базы данных

Немаловажным фактором в задержках является также база данных. Для хранения «горячих данных» вроде токенов пользователя мы будем использовать

Redis, так как он, храня информацию в оперативной памяти, уменьшает задержки памяти с уровня `ssd/hdd` до уровня оперативной памяти, что, исходя из таблицы на рис.7, может снизить задержку доступа к данным на 2-4 порядка.

Для некритичных данных используем **PostgreSQL** ввиду её субъективного удобства и хорошего потенциала к масштабируемости, а для файлов используем S3 хранилище **Ceph**.

Следует пояснить, почему выбрано именно S3 хранилище для файлов вместо обычной файловой системы. Даже если бы у нас была не распределенная архитектура с одним сервером-монолитом, это было бы невероятно неудобно, ненадежно и нестабильно. Файловая система `ext4`, являющаяся системой по умолчанию, не справляется с большим количеством (больше миллиона) директорий и файлов, что было выявлено в «ООО КОМЭКСП» экспериментально. К тому же, обращение к файлам по их путям в коде достаточно неудобно. В свою очередь, **Ceph** имеет в себе оптимизации для хранения множества мелких файлов.

1.8.5. Брокер сообщений

Учитывая распределённую архитектуру системы понадобится также брокер сообщений. Большинство брокеров используют под капотом быстрый протокол **NATS**, являющийся надстройкой над **QUIC**, так что мы возьмем брокер с минимальным оверхедом – **NATS Jetstream**. Он характеризуется минимальными задержками, выдавая 1-5мс с гарантией доставки против 10-50мс у `kafka` [7].

1.8.6. Фронтенд

В качестве фреймворка для фронтенда выберем **React** так как он фактически на данный момент является монополистом рынка среди других фреймворков [8] и предлагает интеграцию с **WASM**. Для ускорения передачи видео будет использоваться **ffmpeg**, скомпилированный в **WebAssembly** для отделения аудио от видео.

1.9. Анализ существующей инфраструктуры

В силу действующего контракта между «ООО КОМЭКСП» и исполнителем данной работы будет рассмотрена только структура существующей базы данных.

На рисунке 8 представлена ER-диаграмма существующей базы данных. Из нее видно, что каждый архив связан с пользователем, а каждое видео — с архивом. На данный момент система позволяет объединять несколько видео в одну бизнес-сущность — архив.

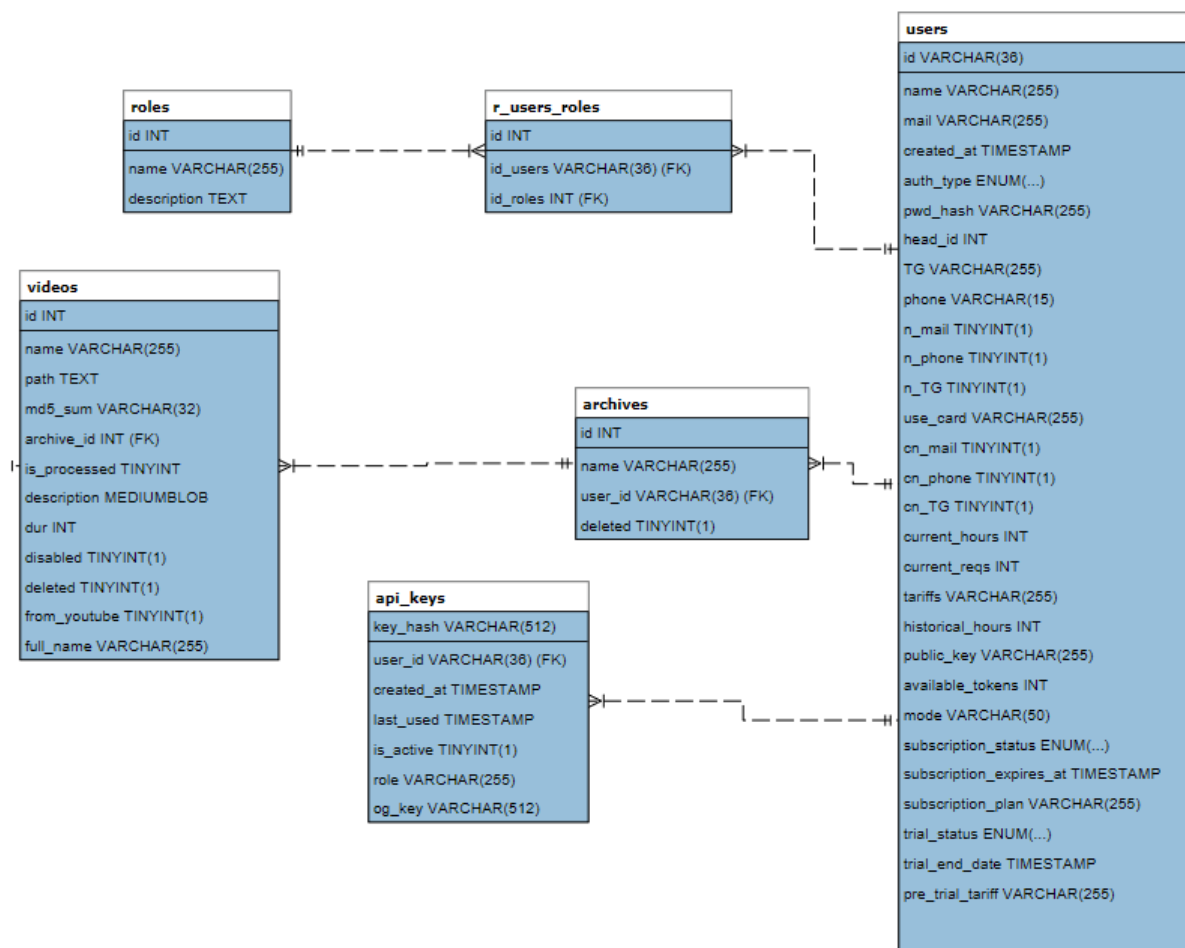


Рисунок 8 – ERD базы данных

1.10. Результаты анализа объекта автоматизации

В ходе работы была проанализирована предметная область, представляющая из себя способы кодирования видео, основные методы оптимизации и краткий разбор технологий, которые будут использоваться. Также были перечислены потенциальные узкие места в стандарте видеокodeка AV1 и основные методы оптимизации согласно профессору Массачусетского Технологического Института Джону Луису Бентли.

Данная информация в будущем поможет нам при проектировании и реализации системы для декодирования видео.

2. ПРОЕКТИРОВАНИЕ

В данной главе будет представлено проектирование системы для декодирования видео в формате AV1. Несмотря на то, что система предназначена только для одной функции, необходимо будет реализовать вокруг неё полноценную инфраструктуру, включающую в себя базу данных, брокер сообщений, фронтенд и бэкенд.

Для последующего анализа эффективности оптимизация также будет спроектирована наивная реализация системы, без каких-либо оптимизаций. Анализ производительности будет производиться по следующим параметрам:

1. Общая задержка от начала отправки видео на сервер до полного декодирования видео, в секундах.
2. Минимальное время декодирования видео без учета времени передачи видео по результатам пяти тестовых запусков.
3. Покадровая метрика потери качества видео, вычисляемая как разница Luma значений каждого пикселя кадра видео.
4. Покадровая метрика PSNR.
5. Время ответа при нагрузочном тестировании системы.

Целевыми метриками считаются ускорение метрик 1 и 2 на 30-50% и 20-35% соответственно в сравнении с наивной реализацией, при этом PSNR должно сохраняться выше 15дб. Метрики 3 и 5 необходимы для характеристики системы в целом, но не критичны.

2.1. Проектирование наивной реализации системы

Наивная реализация системы будет спроектирована максимально просто. Она будет представлять из себя простое Flask-приложение, также включающее в себя frontend как jinja templates. На рис. 9 представлена диаграмма контейнеров наивной системы. Как можно увидеть, центральным является контейнер с flask приложением, включающий в себя весь функционал. Рассмотрим данный контейнер детальнее на рис. 10.



Рисунок 9 – naive system container diagram

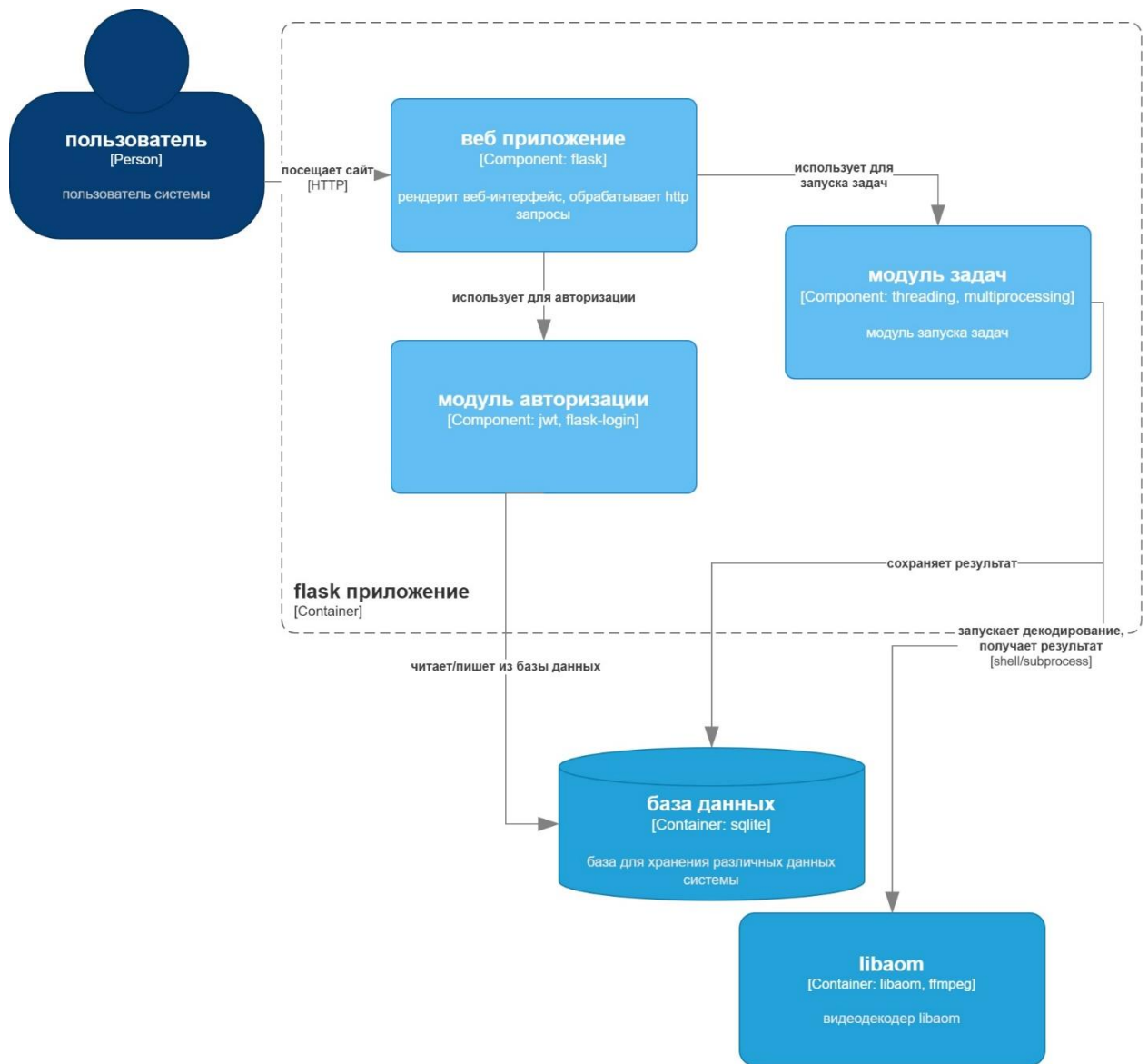


Рисунок 10 – диаграмма контейнера flask приложения

На данной диаграмме показано, как контейнер будет выглядеть внутри. Он будет использовать обыкновенную jwt-систему для авторизации, токены при этом будут храниться в sqlite. Планирование задач декодирования будет выполнено с помощью встроенных в python3 средств, таких как threading и multiprocessing.

2.1.1. Проектирование базы данных

Учитывая, что наивная реализация необходима только для сравнения с оптимизированной реализацией, оставим только необходимые поля таблиц. В результате получим **следующую** ER диаграмму, представленную на рисунке 11. В данной схеме есть 4 таблицы.

Таблица Users предназначена для хранения информации о пользователях, идентификация производится по номеру телефона и паролю, притом пароль не хранится в открытом виде.

Таблица `archives` является бизнес-сущностью, объединяющей несколько видео. У архива есть название и идентификатор, притом архив может принадлежать одному пользователю.

Таблица `Videos` хранит метаданные о видео, включая его путь в файловой системе на сервере, md5 сумму и состояние, в котором оно находится.

Таблица `Api_keys` хранит в себе токены для авторизации пользователя.

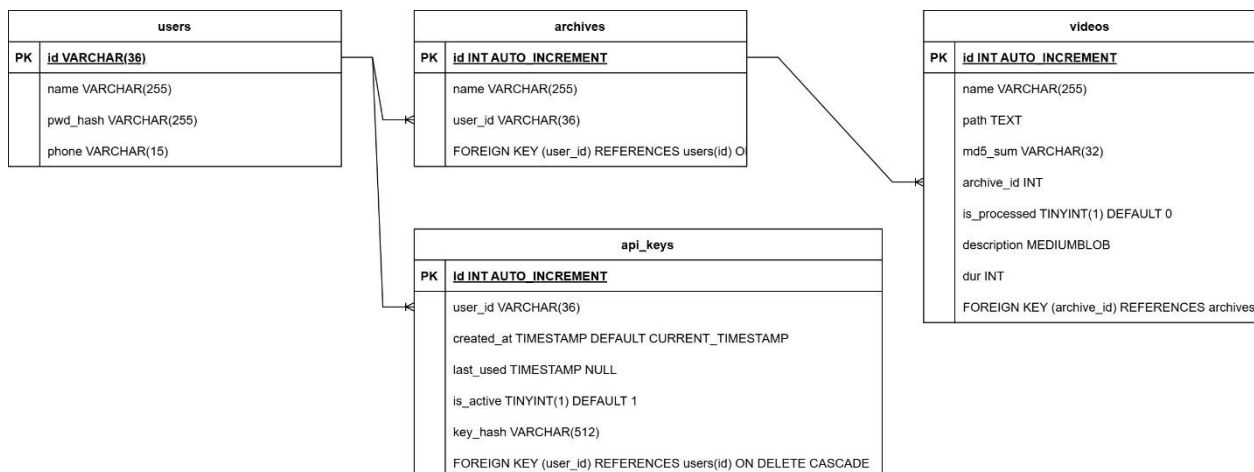


Рисунок 11 – ER диаграмма базы данных наивной реализации

2.2. Проектирование оптимизированной реализации системы

Наивная архитектура, рассмотренная выше, представляет из себя неоптимизированную монолитную систему, которую крайне сложно масштабировать. В данной главе будет рассмотрена реализация максимально быстрой и надежной версии системы.

Прежде всего нам необходимо будет сократить ИО издержки, связанные с передачей файлов по сети. Учитывая, что система необходима для поиска видео по видео, объем файлов в некоторых случаях может достигать двух гигабайт. Чтобы минимизировать задержку, вспомним, что для декодирования видео не нужна звуковая дорожка, присутствующая почти в каждом видеофайле. В связи с этим нам понадобится на стороне клиента вырезать из видеофайла аудиодорожку и отправлять в бэкенд получившийся файл. Также мы будем использовать React как фреймворк для фронтенда ввиду его популярности и что более важно, масштабируемости.

Как было показано на рис. 7, длительность обращения к оперативной памяти может быть быстрее обращения к памяти на жестком диске в тысячу раз, это означает, что если мы сохраним видео в файловую систему и будем декодировать его оттуда, то это будет сильно медленнее чем декодирование из оперативной памяти. Для минимизации данной задержки используем принцип *memory buffer*, то есть декодирование из оперативной памяти напрямую. Данный подход работает не всегда, особенно при высоких нагрузках, и в этом случае всё-таки понадобится сохранить файл. Учитывая нашу распределённую архитектуру, используем S3 хранилище. Оно необходимо, учитывая, что нам нужно будет распределить хранилище по нескольким серверам, что решит проблему ограниченности памяти.

Backend же тоже будет распределён по нескольким серверам. Для того, чтобы обеспечить согласованность серверов и доставку сообщений будет использован брокер сообщений NATS Jetstream. На таблице 1 продемонстрировано сравнение популярных брокеров сообщений. Стоит отметить, что очередь необходима нам также для асинхронных коммуникаций, а потому необходима персистентность сообщений. Стоит отметить, что NATS Jetstream – не идеальный брокер, который, в отличие от Apache Kafka, появился сравнительно недавно и поэтому у некоторых пользователей Jetstream появляются проблемы со стабильностью. Тем не менее, это один из самых быстрых брокеров, что имеет первостепенное значение для решаемой задачи.

Таблица 1 – сравнительный анализ брокеров сообщений

Брокер сообщений	Латентность	Сравнительная пропускная способность	Персистентность
Core NATS	<1мс [13]	Высокая	Нет
NATS JetStream	1-5мс [12]	Выше среднего	Да
Apache Kafka	10-50мс [12]	Высокая	Да
RabbitMQ	5-20мс [14]	Средняя	Да

Система представлена в виде диаграммы С4 на рис. 12.

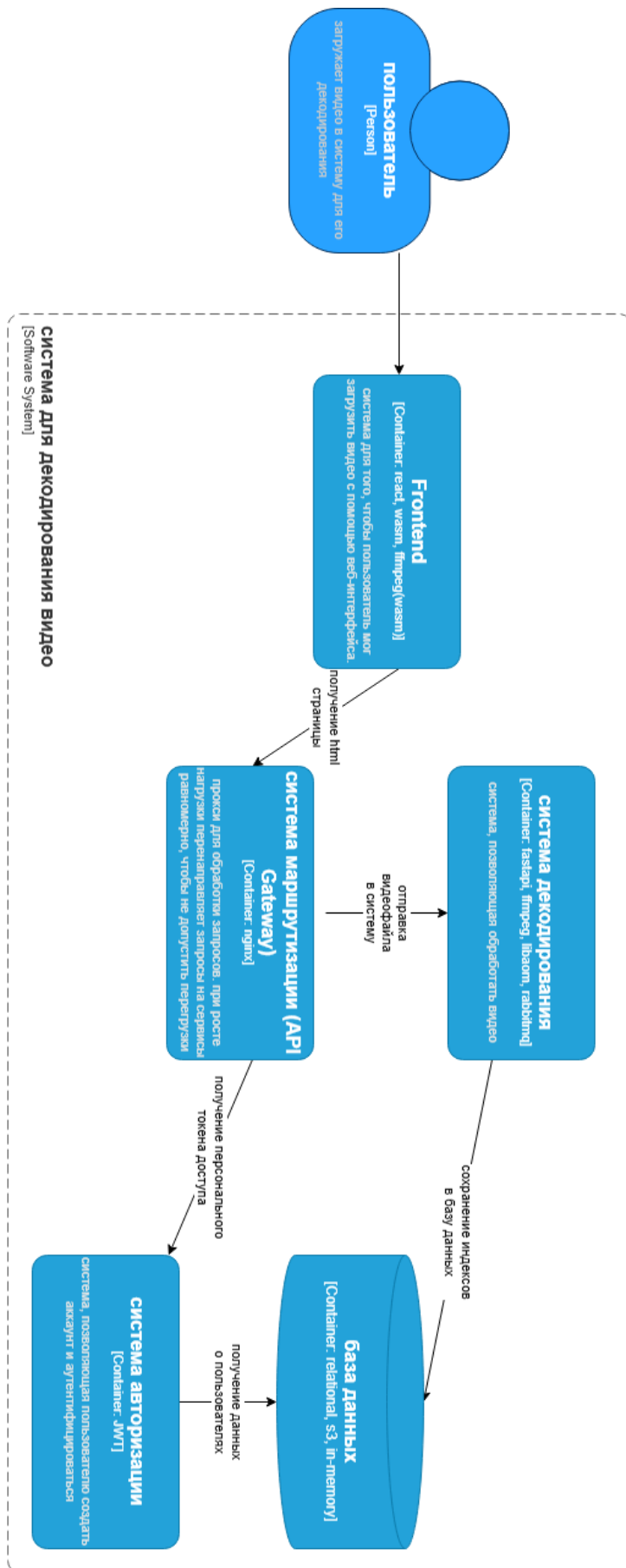


Рис. 12 – диаграмма контейнеров системы.

Диаграмма компонентов представлена на рис. 13. Рассмотрим самые важные из них в следующих главах.

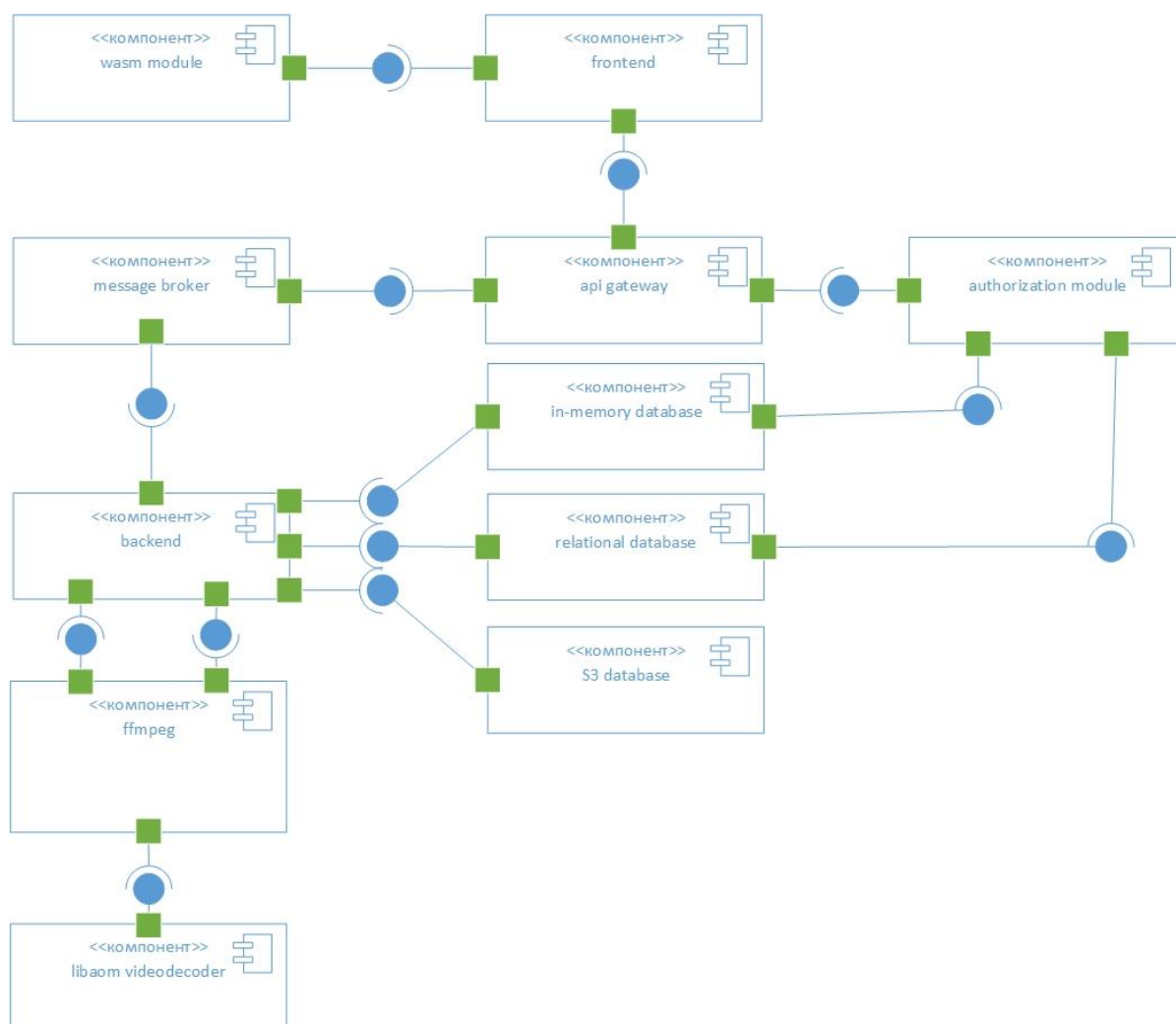


Рисунок 13 – диаграмма компонентов оптимизированной системы

2.2.1. Проектирование реляционной базы данных

Для данной системы будут использоваться несколько различных баз: low-latency база для горячих данных, реляционная база для стандартных реляционных данных и S3 хранилище Serp для видеофайлов и индексов.

Опустим проектирование нереляционных баз ввиду того, что их структура легче поддаётся изменению.

В качестве реляционной базы будет использован PostgreSQL ввиду его масштабируемости и интеграции, а также более высокой устойчивостью к высоким нагрузкам чем у его прямого конкурента MySQL.

Структура таблиц будет перенесена из существующей базы, ER диаграмма который представлена на рис. 8, так что она будет оставлена без изменений. Имеет смысл рассмотреть предназначение таблиц в существующей базе данных.

Таблица Users, как и в наивной реализации, хранит данные о пользователях, но в гораздо большем объеме, включая метод авторизации, дату регистрации, тарифный план и т.д.

Таблица Roles и соответствующая ей таблица r_users_roles позволяют установить соответствие конкретного пользователя/пользователей к роли/ролям, будь то администратор или обычный пользователь.

Таблицы Videos и Api_keys хранят информацию, аналогичную тем же таблицам в наивной реализации.

2.2.2. Проектирование бэкенда

Бэкенд системы является центральным компонентом всей архитектуры и отвечает за обработку пользовательских запросов, управление задачами декодирования, взаимодействие с базами данных и интеграцию с видеodecoderом. Основной задачей серверной части является обеспечение максимально быстрой обработки операций «загрузка – декодирование» при сохранении масштабируемости системы. Для этого был выбран подход горизонтального масштабирования, при котором увеличение вычислительных мощностей достигается за счет увеличения серверов. Компонент Api Gateway проксирует запросы на свободный сервер-реплику с компонентами бэкенда и декодера, а если такового нет, то на наименее занятый сервер. Степень загрузки мониторится из бэкенда и сохраняется в redis.

Для того, чтобы обработать все запросы, будет применен паттерн «request queue», реализация которого в архитектуре системы представлена на рис. 14. Смысл данного паттерна в том, чтобы сохранять запросы в очередь и равномерно маршрутизировать их на реплики бэкенда, что гарантирует обработку всех запросов.

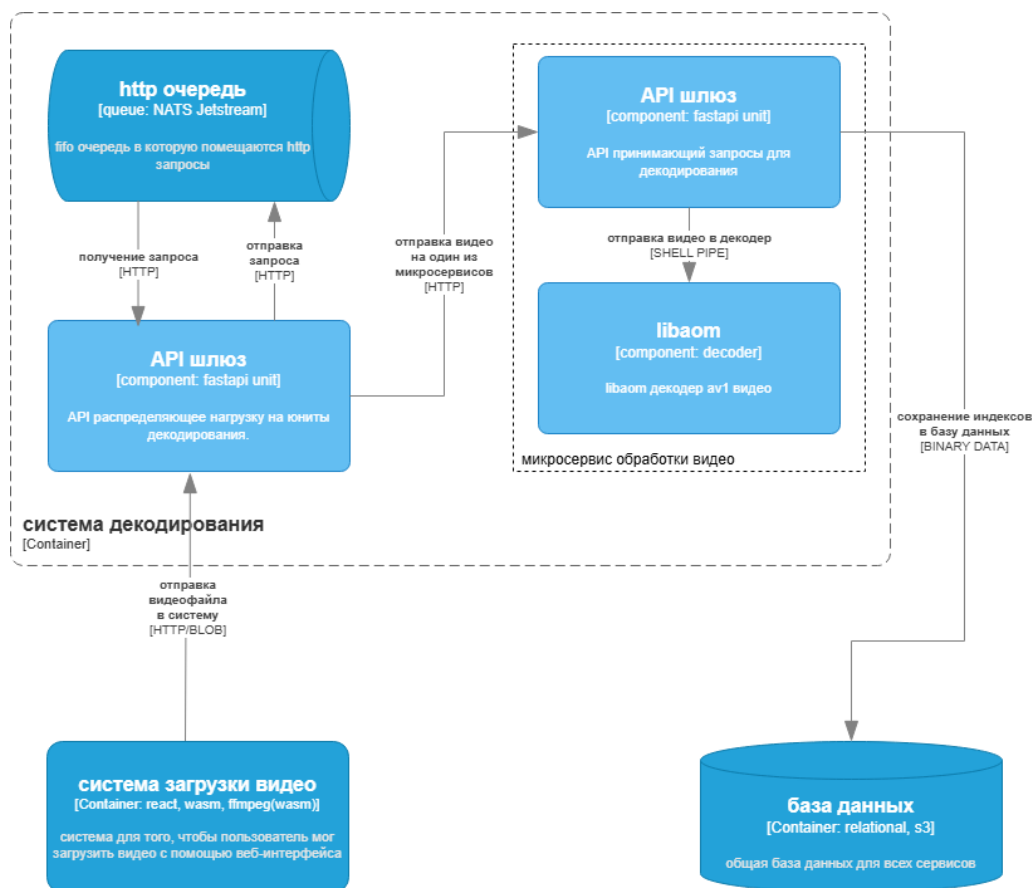


Рис. 14 – Схема системы декодирования

Как видно из рис. 13, бэкенд связан с компонентами message broker, api gateway и ffmpeg. FFMPEG является инструментом с открытым кодом для работы с видео и де-факто стандартом видеоиндустрии. В нём реализованы методы работы с файловой системой и видеodeкодерами, поэтому и в наивной, и в оптимизированной реализации он будет необходим для использования Libaom. Коммуникация с FFMPEG будет происходить через shell, с использованием in-memory файловой системы или S3 хранилища когда это невозможно. На рис. 15 представлена упрощенная реализация decision tree маршрутизатора. Как можно заметить, наиболее приоритетный сервер для декодирования – сервер со свободной оперативной памятью и вычислительными мощностями. В случае, если у нас не хватает оперативной памяти, будет использован метод потокового декодирования из S3 хранилища. На данный при определении доступной вычислительной мощности для декодирования мы исходим из того, что количество доступных ядер = суммарное количество логических ядер процессора – 1, так как одно из ядер должно оставаться за wsgi сервером. В случае превышения этого количества ожидается снижение производительности за счет вытеснения одного процесса другим. В главе «реализация» будет экспериментально проверено оптимальное количество ядер.

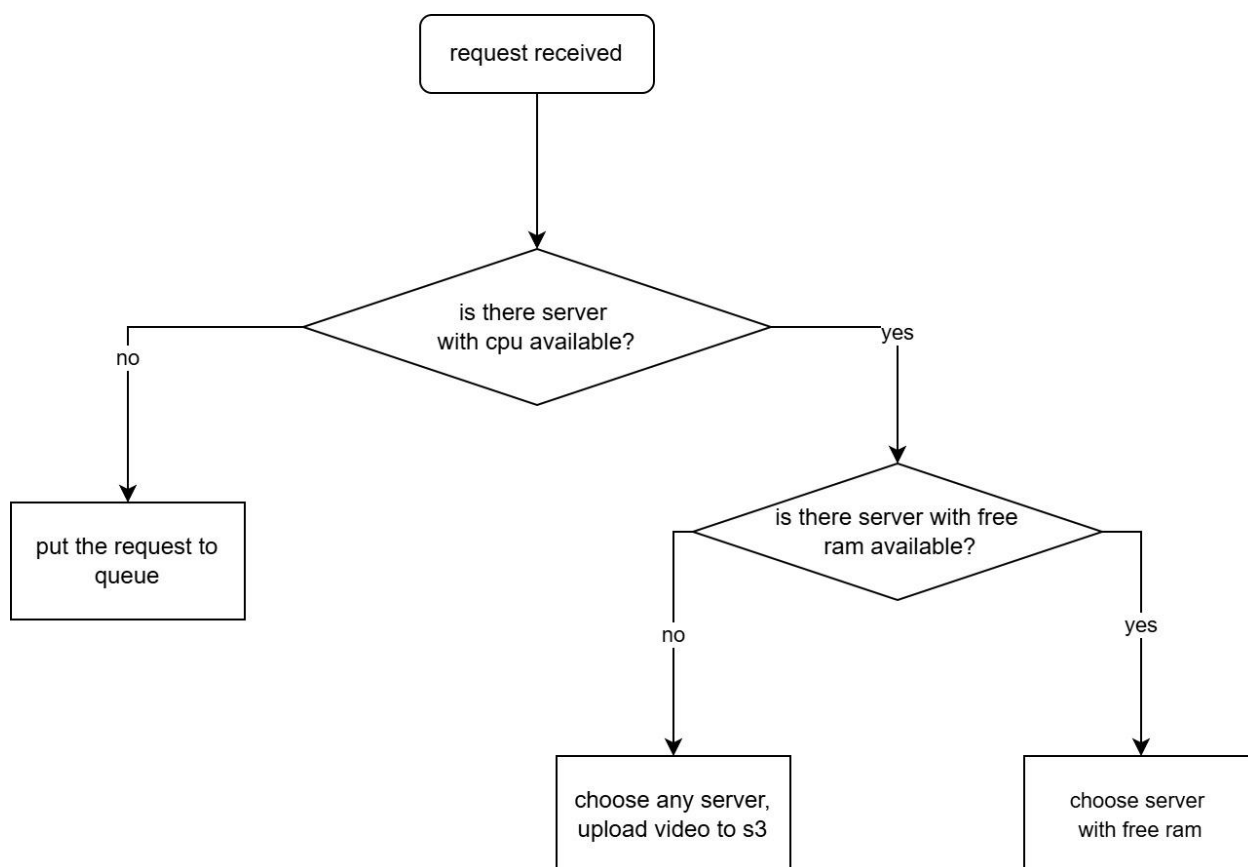


Рис. 15 – Алгоритм метода выбора сервера для декодирования

Как было рассмотрено в главе 2.2.1, каждому видео соответствует статус обработки. Это целое значение, которое имеет несколько состояний: uploaded, queued, processing, done, failed, retrying. Их возможная смена показана на рис. 21.

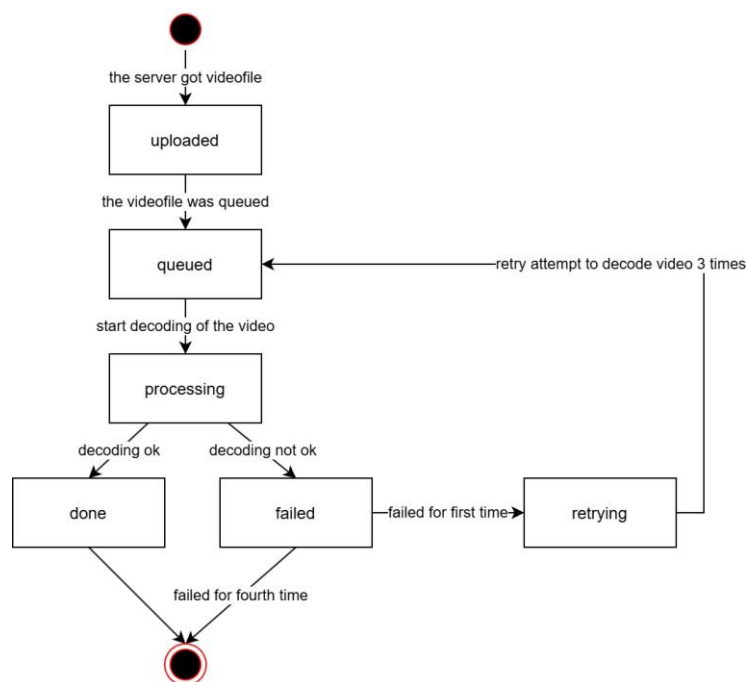


Рис. 21 – диаграмма состояний видео

В таблице 2 показаны требуемые REST эндпоинты, которые будут реализованы в API.

Таблица 2 – требуемые эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ	Код
POST	/auth/register	Регистрация пользователя	Phone, password	user_id	201
POST	/auth/login	Авторизация	Phone, password	access_token, refresh_token	200
POST	/auth/refresh	Обновление токена	Refresh_token	access_token	204
POST	/auth/logout	Выход	-		200
POST	/archives	Создать архив	Name	archive_id	200
GET	/archives	Список архивов пользователя		[{archive_id, name, created_at}]	200
POST	/archives/{archive_id}/videos	Загрузить видео в архив	.obu файл	video_id, task_id	202
GET	/archives/{archive_id}/videos	Список видео в архиве		[{video_id, name, state}]	200

DELETE	/videos/{video_id}	Удалить видео			204
--------	--------------------	---------------	--	--	-----

Выше было упомянуто, что задача имеет свой статус, однако в требуемых эндпоинтах нет опции узнать статус. Это сделано из-за того, что нотификация пользователя будет происходить с помощью соединения через вебсокеты.

Уведомление через вебсокеты – довольно удобное и логичное решение, избавляющее нас от необходимости посылать http запрос каждый несколько секунд чтобы узнать статус. Однако у нас появляется новое ограничение, связанное с фундаментальной особенностью вебсокета: как известно, на каждом экземпляре ОС есть лимит на количество подключений, по умолчанию 1024. Чтобы избежать замедления воспользуемся мощностями серверов, проксируемых с помощью API Gateway. Каждое соединение вебсокета будет держаться на отдельном сервере до момента пока у пользователя есть активные задачи. Когда приходит уведомление о конце задачи в очередь происходит бродкаст-рассылка этого сообщения на все реплики, и реплика, выполняющая эту задачу, закрывает соединение. Это показано на рис. 22.

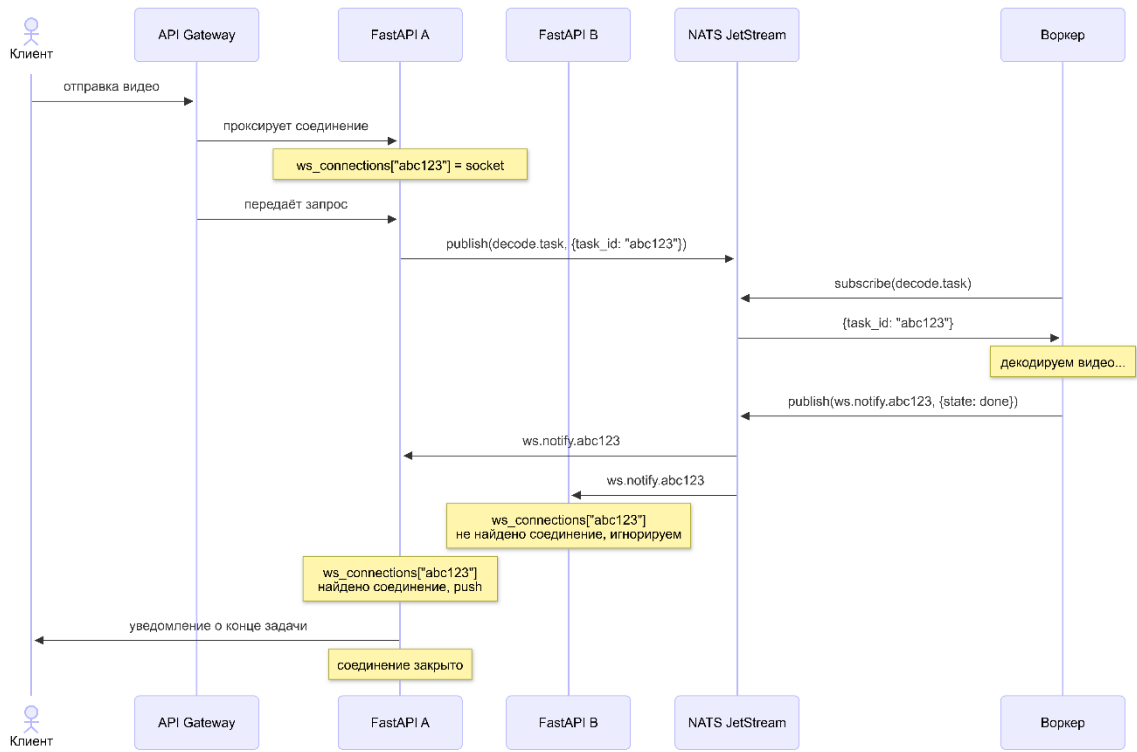


Рисунок 22 – диаграмма последовательности для уведомления клиента

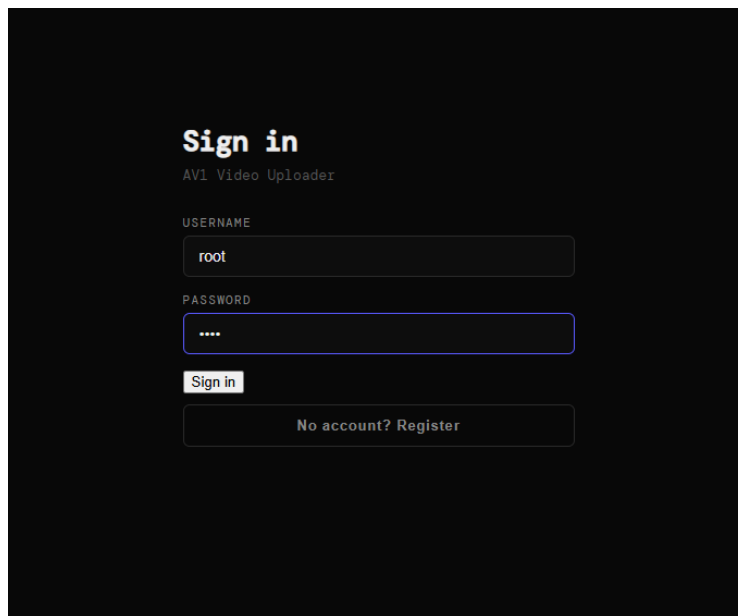
2.2.3. Проектирование фронтенда

Фронтенд системы выполняет несколько задач, включающих в себя авторизацию пользователя, загрузку видеофайлов, их предварительную обработку и отправку обработанных видео на сервер.

Для отделения аудиоданных от видео используется FFMPEG, скомпилированный под архитектуру WebAssembly. Компиляция осуществляется с

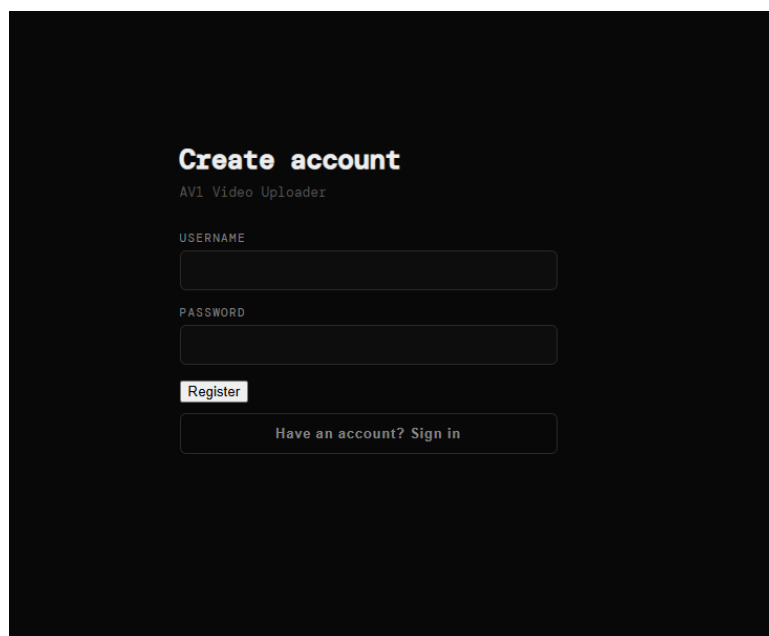
помощью инструмента Emscripten, что позволит запустить код на C в браузере, на стороне клиента.

Дизайн не играет особой роли в данной системе (по крайней мере, не является основной целью), так что было принято решение спроектировать его максимально простым. Интерфейс состоит из трех экранов: регистрация, логин и загрузка видео. Их прототипы представлены на рис. 16, 17 и 18 соответственно.



The image shows a dark-themed prototype of a 'Sign in' screen. At the top, the text 'Sign in' is displayed in a large, bold, white font. Below it, in a smaller white font, is 'AV1 Video Uploader'. The form consists of two input fields: the first is labeled 'USERNAME' and contains the text 'root'; the second is labeled 'PASSWORD' and contains four dots. Below the password field is a white 'Sign in' button. At the bottom, there is a link that says 'No account? Register'.

Рис. 16 – экран входа



The image shows a dark-themed prototype of a 'Create account' screen. At the top, the text 'Create account' is displayed in a large, bold, white font. Below it, in a smaller white font, is 'AV1 Video Uploader'. The form consists of two input fields: the first is labeled 'USERNAME' and is empty; the second is labeled 'PASSWORD' and is empty. Below the password field is a white 'Register' button. At the bottom, there is a link that says 'Have an account? Sign in'.

Рис. 17 – экран регистрации

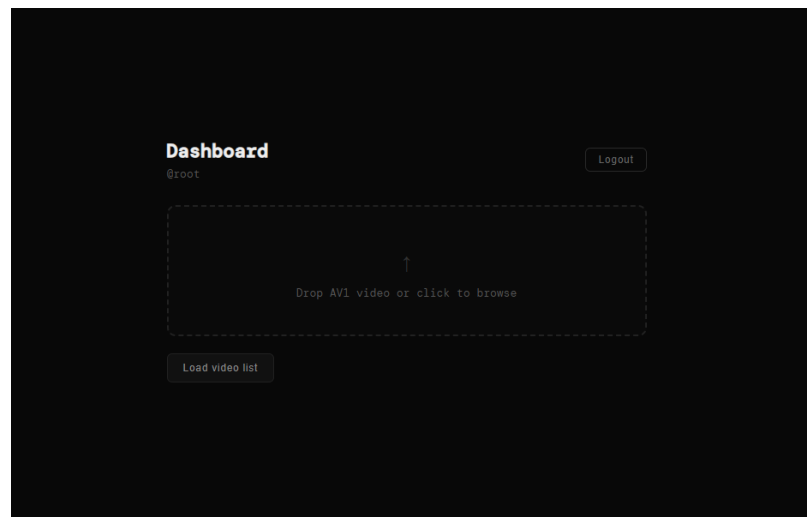


Рис. 18 – экран загрузки видео

2.2.4. Проектирование оптимизаций libaom

В данной главе будут описаны основные методы оптимизации libaom, ранее затронутые в главе 1. Стоит оговориться, что видеodecoder – крайне оптимизированное ПО, и любое серьёзное вмешательство может вызвать в нём эффект домино, сбивая общую структуру, оптимизированную под процессорный кеш, архитектуру процессора и пр. В связи с этим в данной главе не будут предлагаться радикальные вмешательства, меняющие общую структуру данных или логику работы кодера.

CDEF

Constrained directional enhancement filter, как было рассмотрено в главе 1.6.2, является ресурсоемкой но важной частью для сохранения качества видео при декодировании. Тем не менее, в задаче декодирования в сверхнизком разрешении артефакты на границах блоков, как правило, не влияют на качество видео, так что будут проведены два эксперимента: с четырьмя фильтрами с углами наклона (45° , 0° , -45° , -90°) вместо восьми исходных (45° , 22.5° , 0° , -22.5° , -45° , -67.5° , -90° , -112.5°) и с отключённым CDEF.

DF

В отличие от CDEF, DF имеет также высокое значение для низкого разрешения. Это значит, что если отключить фильтр блочности сразу на всех кадрах, то артефакты будут накапливаться от опорных кадров с каждым новым В-кадром. Для устранения этого эффекта будет использоваться подход, при котором фильтр блочности будет применяться только для опорных кадров, хотя стандарт errata и подразумевает что DF должен быть включен либо выключен везде. В таком случае накопление артефактов должно замедлиться, учитывая, что каждый кадр использует сразу несколько опорных кадров. Также будет реализована версия, в которой DF вырезан полностью, чтобы подтвердить или опровергнуть вышеупомянутую гипотезу.

Entropy coding

Теоретически энтропийное декодирование занимает меньше ресурсов, чем интерполяция или фильтры. Тем не менее, оно происходит на каждом кадре в минимум одном блоке, так что имеет смысл попробовать его оптимизировать.

Как упоминалось в главе анализа, DCT и IDCT использует таблицу частот, также называемую таблицей квантизации. В качестве оптимизации можно применить подход обрезки, квантовав её в 4 раза, оставив только частоты в верхнем левом квадрате. На рис. 19 красным цветом обозначены частоты, которые мы хотим оставить, а синим – все доступные частоты при операции IDCT. Если гипотеза верна, то при такой оптимизации видео с низким разрешением почти не потеряет качество, так как высокие частоты используются для кодирования мелких деталей видео, которые нам не важны. В случае успеха оптимизация будет расширена до других методов энтропийного декодирования.

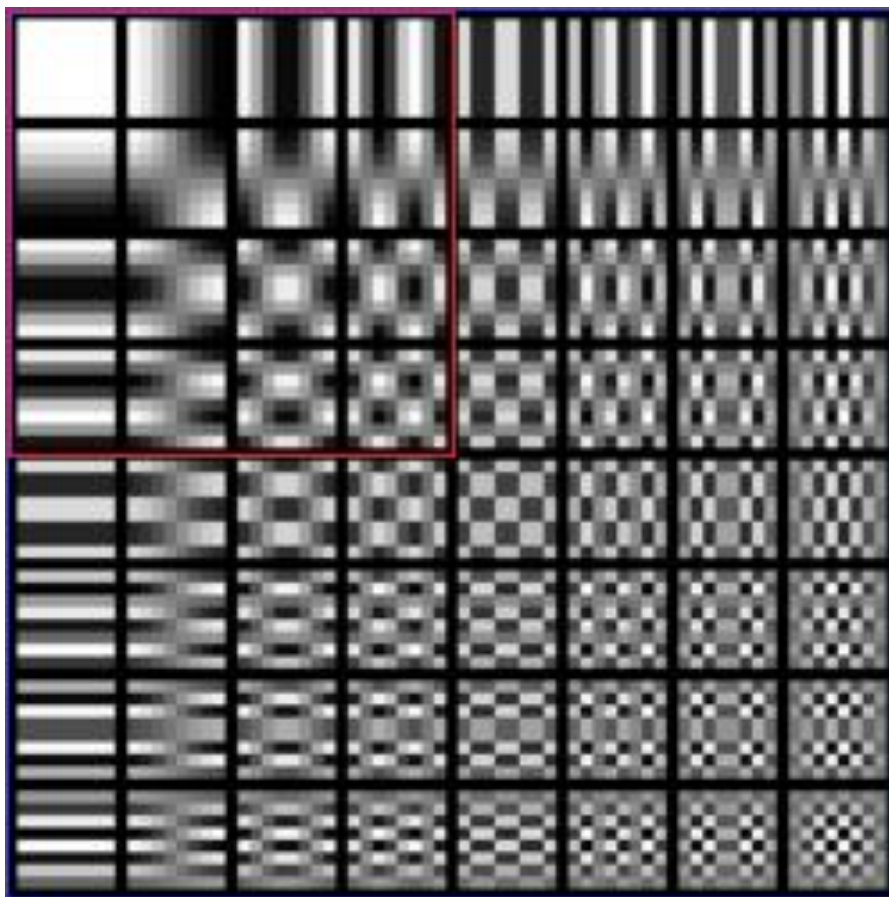


Рис. 19 – объяснение метода квантизации таблицы коэффициентов

Интерполяция

Для оптимизации интерполяции, как и для DF и CDEF будут использоваться 2 метода: полный вырез и максимальное облегчение. Для второго варианта нам нужно предусмотреть сохранение качества, потому что иначе может произойти каскадная деградация качества. На рис. 20 можно заметить, что в процессе интерполяции коэффициенты на 1,2,7 и 8 позициях очень редко имеют большое значение. Это значит, что мы можем попробовать без ущерба для качества видео вырезать их и оставить 4-tap интерполяцию по коэффициентам 3-6, либо 2-tap, в зависимости от метрики PSNR по результату.

Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,0,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-12,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-10,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,-2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,-2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

Рис. 20 – таблица коэффициентов интерполяции [11]

2.3. Выводы

В ходе проектирования была разработана архитектура системы быстрого декодирования видео в формате AV1, удовлетворяющая функциональным и нефункциональным требованиям, сформулированным в главе 1.

Для обеспечения возможности сравнительного анализа была спроектирована наивная реализация системы на основе монолитного Flask-приложения с хранилищем SQLite. Данная реализация намеренно лишена каких-либо оптимизаций и служит базовой точкой отсчёта для измерения эффективности оптимизированной версии.

Оптимизированная реализация системы построена на принципах горизонтального масштабирования и минимизации задержек на каждом уровне стека. В качестве ключевых архитектурных решений были приняты следующие: использование брокера сообщений NATS JetStream для асинхронной маршрутизации задач декодирования между репликами бэкенда; применение Redis для хранения горячих данных о состоянии задач с целью снижения задержки доступа по сравнению с реляционной базой данных; декодирование видео из оперативной памяти в приоритетном порядке, с использованием S3-хранилища Serp в качестве запасного варианта при недостатке памяти.

Спроектирован REST API, необходимый для авторизации, управления архивами и жизненного цикла задач декодирования. Для уведомления клиента о завершении декодирования выбран протокол WebSocket вместо HTTP-поллинга, что исключает избыточные запросы к серверу и снижает итоговую задержку уведомления с 0–2000 мс до уровня сетевой латентности.

Жизненный цикл задачи декодирования формализован в виде диаграммы состояний, включающей шесть состояний: uploaded, queued, processing, done, failed, retrying.

На стороне клиента спроектирован модуль предварительной обработки видео на базе ffmpeg, скомпилированного в WebAssembly, выполняющий отделение аудиодорожки перед отправкой файла на сервер. Данное решение позволяет сократить объём передаваемых данных без потери информации, необходимой для индексации.

Таким образом, спроектированная система обеспечивает выполнение всех требований, выявленных в ходе анализа предметной области, и формирует основу для последующей реализации и оптимизации декодера libaom, описанных в главе 3.

3. РЕАЛИЗАЦИЯ

Тут про реализацию. Нужно будет в основном рассказать про оптимизации т.к. это – главная тема.

1.1. Оптимизации Libaom

3.1.1. Оптимизация 1

Для каждой оптимизации: что сделал и бенчмарк x3 на фильме форсаж + бенчмарк на стоке.

3.1.2. Оптимизация 2

3.1.3. Оптимизация 20

3.2.Реализация backend

Надо будет рассказать про бд, очередь, фастапи, связь с декодером.

3.3.Реализация frontend

Тут про wasm, авторизацию и бенчмарк насколько быстрее стало когда мы начали отделять obu от видео и вырезать звук. Либо просто сравнение запакованного ffmpeg (2mb (до сих пор удивляюсь)) плюс видео без звука vs просто видео. Насколько я помню там -30%. Надо будет сослаться на memory latency table.

3.4.Результаты оптимизации

Сюда надо будет написать время работы в целом в сравнении с максимально стоковой (flask + libaom). Либом у меня на 70% оптимизирован, система думаю - 40% совокупных выдаст.

ЗАКЛЮЧЕНИЕ

Здесь подводятся итоги, даётся оценка результатов выполнения проекта...

Не нужно просто перечислять решённые задачи – нужно охарактеризовать результат решения каждой из них...

В результате анализа деятельности... выявлены проблемы..., причиной которых являются...

Анализ показал, что существующие решения могут..., но не..., имеются ограничения, ...

Результаты анализа требований оформлены в виде технического задания (Ошибка! Источник ссылки не найден.).

В ходе проектирования разработана модель..., ...

При реализации прототипа создано, ... разработаны... Общий объём кода...

Все описанные сценарии реализованы и созданные... протестированы на основе (...).

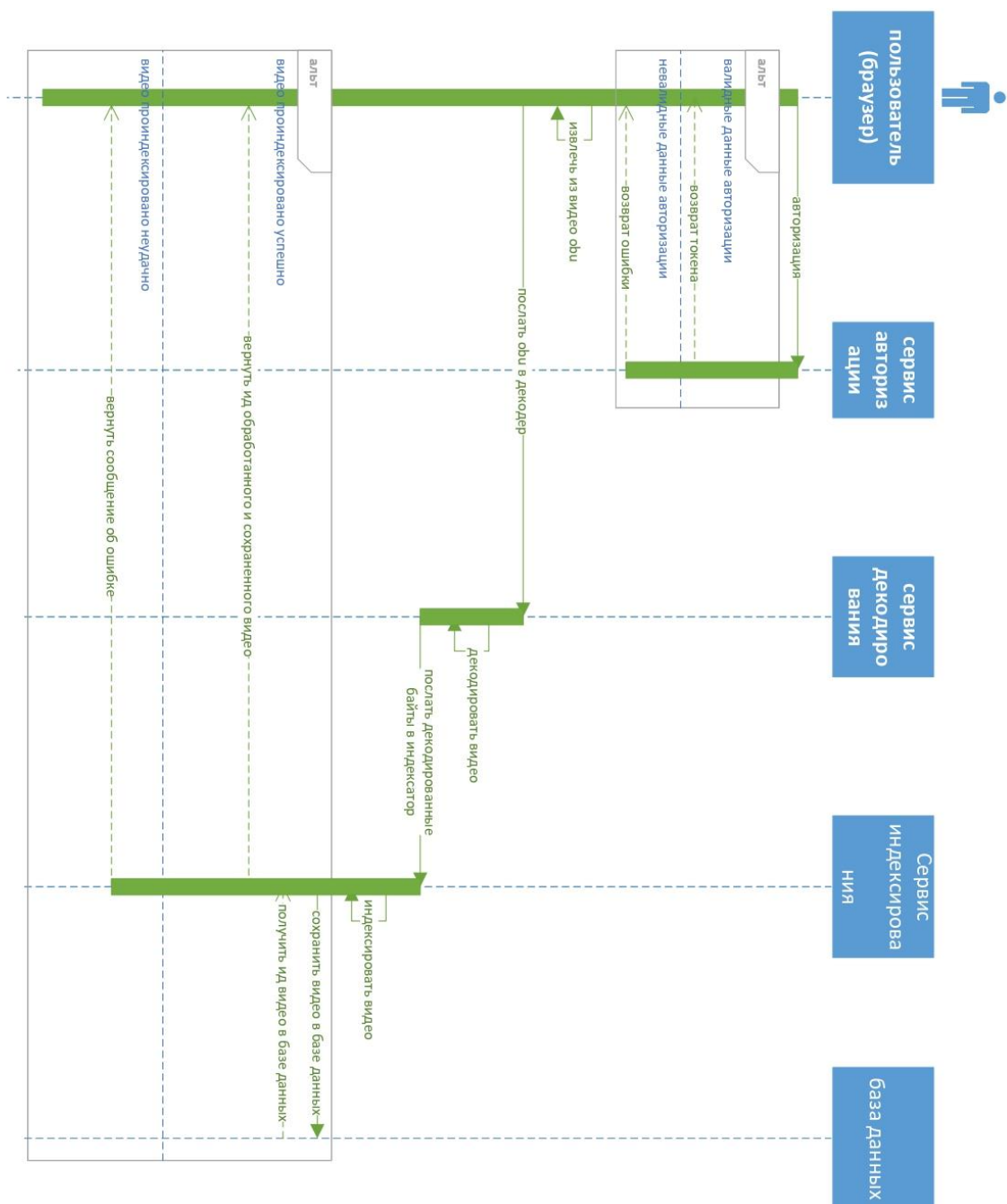
Полученные результаты могут быть полезны, ... использованы...

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Alliance for Open Media. AV1 Bitstream & Decoding Process Specification / Version 1.0.0 with Errata 1 [Электронный ресурс]: технический стандарт / консорциум Alliance for Open Media. – 2021. – URL: <https://aomediacodec.github.io/av1-spec/av1-spec.pdf> (дата обращения: 19.12.2025).
2. Международный союз электросвязи. Сектор стандартизации (ITU-T). *Рекомендация ITU-T H.264 (V15). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services* [Электронный ресурс]. — изд. от 13.08.2024. — (Серия H: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).
3. Yue Chen, An Overview of Core Coding Tools in the AV1 Video Codec / Yue Chen, D. Murherjee, J. Han // IEEE. — P. 41-45 — 2018. — DOI: 10.1109/PCS.2018.8456249.
4. D. Petreski, Next Generation Video Compression Standards – Performance Overview / D. Petreski, T. Kartalov // IEEE. — 2023. — P. 1–5. — DOI: 10.1109/IWSSIP58668.2023.10180261.
5. W. Kolodziejski, Speeding Up the AV1 Global Warped Motion Compensation / W. Kolodziejski, R. Domanski, L. Agostini // IEEE. — 2024. — P. 1-5. — DOI: 10.1109/LASCAS60203.2024.10506160.
6. Y. Yi, High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec / Y. Yi, B. C. Song // IEEE. — 2008. — P. 891–894. — DOI: 10.1109/LSP.2008.2001982.
7. NATS JetStream vs RabbitMQ vs Apache Kafka on VPS in 2025: Throughput/Latency Benchmarks, Exactly-Once vs At-Least-Once, Durability/Replication, TLS/mTLS, and the Best Message Broker for Your Stack / блог компании Onidel [Электронный ресурс]. 8.11.2025. URL: <https://onidel.com/blog/nats-jetstream-rabbitmq-kafka-2025-benchmarks> (дата обращения: 13.02.2026).
8. Angular vs. React vs. Vue.js in 2025 / блог компании DLT Software [Электронный ресурс]. 20.11.2025. URL: <https://medium.com/@dltsoft/angular-vs-react-vs-vue-js-in-2025-f24b56ff0064> (дата обращения: 13.02.2026).
9. H.264 is Magic / Sid Bala [Электронный ресурс]. 2.11.2016. URL: <https://sidbala.com/h-264-is-magic/> (дата обращения: 13.02.2026).
10. Wei-Ting Lin, Efficient AV1 Video Coding Using a Multi-layer Framework / Wei-Ting Lin, Zoe Liu, Debargha Mukherjee // IEEE. — P. 368-369 – 2018. — DOI: 10.1109/DCC.2018.00045.
11. SVT-AV1 documentation: ALTREF and Overlay Pictures [Электронный ресурс]. URL: <https://gitlab.com/AOMediaCodec/SVT-AV1/-/blob/master/Docs/Appendix-Alt-Refs.md> (дата обращения: 13.02.2026).
12. https://www.utupub.fi/bitstream/handle/10024/182402/Salonen_Nino_opinnayte.pdf
13. <https://medium.com/@nathanbcrocker/comparing-nats-nats-jetstream-and-kafka-performance-for-varying-payload-sizes-3538c94ac56c>

14. <https://gcore.com/learning/nats-rabbitmq-nsq-kafka-comparison>
15. <https://dasroot.net/posts/2026/02/python-flask-fastapi-django-framework-comparison-2026/>
- 16.

ПРИЛОЖЕНИЕ А. ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТЕЙ



ПРИЛОЖЕНИЕ Б. ДИАГРАММА ПРЕЦЕДЕНТОВ



ПРИЛОЖЕНИЕ В. ?????

Приложения могут содержать справочные материалы, таблицы большого объёма с описанием классификаторов и пр., а также тестовые сценарии и пр.

В приложения выносятся программная документация (руководство программиста, руководство пользователя, ...администратора...).

B.1. ???

?????

[illegible]

B.2. ???????

?????

?????????? ?????????????????? ?????????????????? ??????????????? ?????????? ??????
 ?????????????????? ?????????????????? ??????????????? ?????????????????? ??????????????? ???????????
 ?????????????????